

คู่มือระบบควบคุมแขนกล Dobot Magician ด้วย ROS 2, OpenCV, pydobot2 และ PyQt

การพัฒนาระบบแขนกลอัจฉริยะ คัดแยกสี หลบหลีกสิ่งกีดขวาง และเชื่อมบลูทูธอัตโนมัติ (เริ่มต้นจากศูนย์)

ผู้พัฒนา / ผู้จัดทำ: Thanadech (Google Deepmind Pair-Programming Assistant)	สภาพแวดล้อม: Ubuntu Linux 22.04 LTS / ROS 2 Humble	วันที่: กันยายน 2026
---	--	----------------------

 **เป้าหมายของเอกสารฉบับนี้**

เอกสารฉบับนี้ถูกเขียนขึ้นเพื่อให้ผู้เริ่มต้นที่ไม่มีพื้นฐานด้านหุ่นยนต์หรือ ROS 2 มาก่อน สามารถเข้าใจหลักการทำงาน สถาปัตยกรรมการสื่อสาร โค้ดคอมพิวเตอร์วิทัศน์ และการควบคุมฮาร์ดแวร์แขนกล Dobot Magician ได้อย่างถ่องแท้ ตั้งแต่ทฤษฎีพื้นฐานจนถึงระบบอัตโนมัติ 3 เฟสจริงในโรงงานอุตสาหกรรม

สารบัญเนื้อหา (Table of Contents)

- บทที่ 1: สถาปัตยกรรมระบบและโครงสร้างสนามทำงาน (System Architecture & Field Layout)
- บทที่ 2: พื้นฐาน ROS 2 จากศูนย์ (ROS 2 Fundamentals from Scratch)
- บทที่ 3: ระบบประมวลผลภาพด้วย OpenCV (Computer Vision & Homography)
- บทที่ 4: การควบคุมแขนกล Dobot Magician ด้วย pydobot2 (Hardware Control)
- บทที่ 5: การสร้างส่วนติดต่อผู้ใช้ด้วย PyQt (GUI Architecture & Multi-Threading)
- บทที่ 6: ภารกิจ 3 เฟส และการหลบหลีกสิ่งกีดขวาง (3-Phase Mission & Collision Avoidance)
- บทที่ 7: คู่มือการใช้งานหน้าต่างโปรแกรม (UI Interface Walkthrough with Screenshots)
- บทที่ 8: คำสั่งเริ่มต้นใช้งานและการแก้ไขปัญหา (Build, Run & Troubleshooting)

บทที่ 1: สถาปัตยกรรมระบบและโครงสร้างสนามทำงาน

1.1 ภาพรวมทางกายภาพของสนามทำงาน (Field Layout)

ระบบนี้ออกแบบมาสำหรับการแข่งขันและปฏิบัติการอัตโนมัติ โดยใช้แผ่นกระดาษขนาด A4 (210 x 297 มม.) เป็นระนาบทำงานหลัก ประกอบด้วยส่วนประกอบสำคัญ 4 ส่วน:

- ฐานแขนกล Dobot Magician:** ติดตั้งอยู่ที่พิกัด ($X = 105.0$ มม., $Y = 270.0$ มม.) ด้านหน้าของสนาม
- ตารางวาง 3x3 (Pallet Grid):** ขนาดภายนอก 114.5 x 114.5 มม. มีระยะห่างระหว่างจุดศูนย์กลางช่อง (Pitch) 35.0 มม. ประกอบด้วย 8 ช่องรอบนอกสำหรับวางบล็อกสีต่าง ๆ
- เป้าหมาย Center Goal (ช่อง [1, 1]):** ช่องตรงกลางของตาราง เป็นจุดที่แขนกลต้องนำบล็อกมาวางซ้อนกันในแนวตั้ง (ถูกบล็อกให้ซ้อนได้สูงสุด 4 ก้อน)
- ช่อง Feeder 4 ตำแหน่ง (Feeder Slots 1..4):** อยู่ทางฝั่งซ้ายของตาราง มีลักษณะเป็นวงกลมสี่เหลี่ยม ใช้เป็นสถานีพักชั่วคราวสำหรับบล็อกที่เป็น "สิ่งกีดขวาง"

```
[4 Feeder Slots]
(Green Rims)
( #1 )
( #2 )
( #3 )
( #4 )

[3x3 Pallet Grid: 114.5 x 114.5 mm]
+-----+-----+-----+
| (0,0) | (0,1) | (0,2) |
+-----+-----+-----+
| (1,0) | (1,1) | (1,2) |
|       | GOAL |       |
+-----+-----+-----+
| (2,0) | (2,1) | (2,2) |
+-----+-----+-----+

[Dobot Magician Robot Base]
(105, 270) mm
```

1.2 สถาปัตยกรรมของซอฟต์แวร์ใน Workspace (dobot_ws)

โปรเจกต์ถูกแบ่งออกเป็น 2 แพคเกจหลักตามหลักการ Single Responsibility Principle:

- dobot_v2** : แพคเกจแกนหลักสำหรับประมวลผลภาพกล้อง (Vision), คำนวณเส้นทางเคลื่อนที่ (Trajectory Generator), ควบคุมมอเตอร์และเครื่องมือปลายแขน (Hardware Driver)
- dobot_ui** : หน้าต่างแสดงบอร์ดกราฟิก พัฒนาด้วย PyQt เชื่อมต่อกับ ROS 2 เพื่อแสดงผลภาพสด แสดงพิกัดสถานะแบบเรียลไทม์ และส่งคำสั่งควบคุมหุ่นยนต์

บทที่ 2: พื้นฐาน ROS 2 จากศูนย์ (ROS 2 Fundamentals)

2.1 ROS 2 คืออะไร?

ROS 2 (Robot Operating System 2) ไม่ใช่ระบบปฏิบัติการที่ลงกับคอมพิวเตอร์เหมือน Windows แต่เป็น *Middleware* หรือกรอบการพัฒนาซอฟต์แวร์หุ่นยนต์แบบกระจายศูนย์ (Distributed System) ที่ช่วยให้โปรแกรมย่อยหลาย ๆ โปรแกรมสามารถคุยกัน รับส่งข้อมูลภาพ พิกัด และคำสั่งได้อย่างรวดเร็วและปลอดภัย ผ่านโปรโตคอลมาตรฐานอุตสาหกรรมที่เรียกว่า **DDS (Data Distribution Service)**

2.2 องค์ประกอบพื้นฐานใน ROS 2

1. Node (โหนด)

เปรียบเสมือน "พนักงานหนึ่งคน" ในโรงงานที่มีหน้าที่ชัดเจนเพียงอย่างเดียว การแยกโปรแกรมเป็นโหนดเดี่ยว ๆ ช่วยให้หากโหนดใดโหนดหนึ่งมีปัญหา โหนดอื่นจะไม่พัวพันไปด้วย:

- detection_node** : ดึงภาพจากกล้อง แปลงพิกัด และส่งรายชื่อบล็อกที่ตรวจจับได้
- dobot_controller** : ติดต่อกับแขนกล Dobot รับคำสั่งภารกิจ และสั่งเคลื่อนที่
- dobot_ui_node** : วาดหน้าจอ GUI รับคลิกจากผู้ใช้

2. Topic (หัวข้อการรับส่งข้อมูล) และ Publisher / Subscriber

เป็นการสื่อสารแบบทางเดียว (One-Way Streaming) แบบ **Publish-Subscribe**:

- Publisher (ผู้ส่ง)**: ส่งข้อมูล (Message) เข้ามายังชื่อ Topic อย่างสม่ำเสมอ
- Subscriber (ผู้รับ)**: ดักฟังสัญญาณ เมื่อมีข้อมูลใหม่เข้ามา Callback Function จะถูกเรียกทำงานทันที

ชื่อ Topic	ประเภทข้อมูล (Message Type)	ผู้ส่ง (Publisher)	ผู้รับ (Subscriber)	คำอธิบาย
/detected_objects	std_msgs/msg/String (JSON)	detection_node	Dobot_UI , dobot_controller	ส่งข้อมูลพิกัด (X, Y, Z) สี และ ตำแหน่งตารางของ บล็อกทั้งหมด
/detected_objects_image/compressed	sensor_msgs/msg/CompressedImage	detection_node	Dobot_UI	สตรีมภาพวิดีโอแบบ บีบอัด JPEG เพื่อ แสดงผลบนหน้าจอ GUI แบบ ความหน่วงต่ำ
/dobot_status	std_msgs/msg/String (JSON)	dobot_controller	Dobot_UI	สถานะแขนกล (IDLE/MOVING), พิกัดปัจจุบัน X, Y, Z, R และสถานะสม ดุด
/dobot_ui_cmd	std_msgs/msg/String (JSON)	Dobot_UI	dobot_controller	ส่งคำสั่งจากปุ่มกด เช่น เริ่มภารกิจ (mission), จ็อก มือ (jog), โฮมมิ่ง (home)

บทที่ 3: ระบบประมวลผลภาพด้วย OpenCV (Computer Vision)

3.1 การจับภาพจากกล้อง (V4L2 Video Streaming)

ในระบบ Linux กล้อง USB จะถูกมองเป็นอุปกรณ์ใน `/dev/video*` โดย OpenCV สามารถเชื่อมต่อผ่านไดรเวอร์ระดับเคอร์เนล **V4L2 (Video4Linux2)** ร่วมกับ การถอดรหัสบีบอัดภาพฮาร์ดแวร์ MJPG ทำให้สามารถสตรีมภาพความละเอียด 640 x 480 ได้ที่ 30 เฟรมต่อวินาที (FPS) โดยไม่หน่วง CPU:

```
import cv2

cap = cv2.VideoCapture('/dev/video0', cv2.CAP_V4L2)
cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
cap.set(cv2.CAP_PROP_FPS, 30)
ret, frame = cap.read() # frame เป็น NumPy Array ในรูปแบบ BGR
```

3.2 ทำไมต้องใช้พื้นที่สี HSV แทน RGB/BGR ในการตรวจจับวัตถุ?

กล้องทั่วไปให้ภาพในระบบ **BGR (Blue, Green, Red)** ซึ่งทั้ง 3 ค่าจะแปรผันตามความสว่างของแสงในห้อง หากแสงแดดส่องเข้ามา หรือมีเงาตกกระทบ ค่า R, G, B จะเปลี่ยนไปทั้งหมด ทำให้การเขียนเงื่อนไขตรวจจับสีล้มเหลว

การแก้ปัญหาคือแปลงภาพเข้าสู่พื้นที่สี **HSV (Hue, Saturation, Value)**:

- **H (Hue - เนื้อสี):** มีค่าตั้งแต่ 0 ถึง 180 ใน OpenCV ระบุเฉดสีแท้จริง เช่น สีแดง (~0-10 และ 160-180), สีเหลือง (~20-35), สีเขียว (~35-85), สีฟ้า (~90-130) ค่านี้จะคงที่เสมอแม้แสงจะเปลี่ยน
- **S (Saturation - ความสดของสี):** แยกวัตถุสีเข้มออกจากสีขาวและแสงสะท้อน
- **V (Value - ความสว่าง):** แสดงความเข้มของแสง สามารถกำหนดขอบเขตต่ำสุดเพื่อตัดเงาตัวออกไปได้

3.3 การแยกสี (Color Masking) และ Morphology

เราใช้คำสั่ง `cv2.inRange()` เพื่อสร้าง Mask ไบนารี (พิกเซลที่อยู่ในช่วงสีที่ต้องการจะเป็นสีขาว 255 และส่วนอื่นเป็นสีดำ 0) จากนั้นใช้การแปลงสัญญาณวิทยา (Morphology) เพื่อลบสัญญาณรบกวน:

```
# แปลงสี BGR เป็น HSV
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

# ตัวอย่างช่วงสีเขียว
lower_green = np.array([35, 80, 70])
upper_green = np.array([85, 255, 255])
mask = cv2.inRange(hsv, lower_green, upper_green)

# ลบ Noise ที่ติดมาด้วย Morphological Opening (Erode แล้ว Dilate)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
mask_clean = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
```

3.4 การแปลงภาพเป็นพิกัดจริงด้วย Perspective Homography

เนื่องจากกล้องติดตั้งทำมุมเอียงกับโต๊ะ ภาพที่ได้จะเอียงเป็นสี่เหลี่ยมคางหมู เราจึงใช้คณิตศาสตร์ **Homography Matrix (3x3)** เพื่อบิดภาพระนาบเอียงให้กลายเป็นระบบตั้งฉากสมบูรณ์:

1. ตรวจสอบจุดมุม 4 จุดของตารางกริดบนภาพกล้อง (u, v เป็นพิกเซล)
2. จับคู่กับพิกัดจริงของกระดาษตาราง 114.5 x 114.5 มม. (x, y เป็นมิลลิเมตร)
3. คำนวณเมทริกซ์การแปลงด้วย `cv2.getPerspectiveTransform()`
4. นำพิกัดพิกเซลของบล็อกทุกก้อนมาคูณเมทริกซ์นี้ เพื่อได้พิกัดจริงบนสนาม และแปลงเข้าสู่พิกัดฐานแกนกลาง (rx, ry, rz) ได้อย่างแม่นยำระดับมิลลิเมตร!

บทที่ 4: การควบคุมแขนกล Dobot Magician ด้วย pydobot2

4.1 การสื่อสารแบบอนุกรม (Serial UART Protocol)

แขนกล Dobot Magician เชื่อมต่อกับคอมพิวเตอร์ผ่านสาย USB โดยภายในมีชิปแปลง USB-to-UART สื่อสารที่ Baudrate **115200 bps** โดยโปรโตคอลของ Dobot จะส่งแพ็คเกจข้อมูลขนาดคงที่ มี Header 0xAA 0xAA, ความยาวข้อมูล (Length), รหัสคำสั่ง (Command ID), ข้อมูลคำสั่ง (Payload) และ Checksum ไตรสารี pydobot2 ทำหน้าที่เป็น Wrapper แปลงคำสั่ง Python ให้กลายเป็นไบนารีแพ็คเกจที่ส่งโดยอัตโนมัติ

4.2 ระบบพิกัด Cartesian (X, Y, Z, R)

ระบบแกนพิกัดของ Dobot Magician

- **X (มม.):** แกนหน้า-หลัง (ยื่นแขนไปข้างหน้าเป็นบวก, ดึงเข้าหาตัวเป็นลบ) ขอบเขตปลอดภัย: 100 ถึง 330 มม.
- **Y (มม.):** แกนซ้าย-ขวา (หมุนไปทางซ้ายเป็นบวก, หมุนไปทางขวาเป็นลบ) ขอบเขต: -250 ถึง +250 มม.
- **Z (มม.):** แกนแนวตั้ง (ยกแขนขึ้นด้านบนเป็นบวก, กดลงต่ำเป็นลบ) ขอบเขต: -60 ถึง +160 มม.
- **R (องศา):** มุมหมุนเซอร์โวมอเตอร์หัวดูดปลายแขน ขอบเขต: -180° ถึง +180°

4.3 โหมดการเคลื่อนที่แบบ PTP: MOVJ vs MOVL

- **MOVJ_XYZ (Joint Interpolation):** มอเตอร์ทุกข้อต่อหมุนพร้อมกัน แขนจะเหวี่ยงเป็นเส้นโค้งอิสระด้วยความเร็วสูงสุด ใช้สำหรับการบินเดินทางในที่โล่ง (Transit)
- **MOVL_XYZ (Linear Interpolation):** หัวดูดปลายแขนจะวิ่งเป็นเส้นตรงบริสุทธิ์ เหมาะสำหรับจังหวะลดระดับลงไปดูดบล็อก (ลงตรง 90°) และยกบล็อกขึ้น เพื่อป้องกันไม่ให้หัวดูดหรือบล็อกเฉี่ยวชนบล็อกก่อนข้างเคียง

```
from pydobot import Dobot
from pydobot.dobot import MODE_PTP

device = Dobot(port='/dev/ttyUSB0')
device.speed(velocity=150.0, acceleration=150.0)

# ยื่นแขน MOVJ ไปยังพิกัดเหนือบล็อก
device.move_to(200.0, 0.0, 80.0, 0.0, mode=MODE_PTP.MOVJ_XYZ)

# ลดระดับลงแนวตั้งแบบเส้นตรง MOVL ไปดูดบล็อก
device.suck(True)
device.move_to(200.0, 0.0, 12.5, 0.0, mode=MODE_PTP.MOVL_XYZ)
```

บทที่ 5: การพัฒนาหน้าต่างส่วนติดต่อผู้ใช้ด้วย PyQt

5.1 สถาปัตยกรรม PyQt (Event Loop & Multi-Threading)

หน้าต่างโปรแกรม GUI ทำงานด้วย **Event Loop** ที่คอยดักจับการคลิกเมาส์และวาดหน้าจอ หากมีงานที่ใช้เวลานาน (เช่น คำนวณภาพ หรือรอการเคลื่อนที่ของแขนกล) มารันบน Main Thread จะทำให้หน้าต่างโปรแกรม "ค้าง" (GUI Freezing) กันที

วิธีแก้ปัญหาในโปรแกรมนี้: เราแยกการทำงานออกเป็น 2 Threads ผ่านคลาส RosBridge(QThread) :

1. **Main UI Thread:** ดูแลการวาดกราฟิก รับคลิกผู้ใช้ และแสดงผลลัพธ์
2. **ROS Bridge Worker Thread ('QThread'):** ทำงานเบื้องหลัง คอยรับคำสั่ง `rcipy.spin_once()` เมื่อมีภาพกล้องหรือสถานะหุ่นยนต์ส่งเข้ามา จะส่งผ่าน `'pyqtSignal'` ข้ามเธรดมาสะกดให้ Main Thread วาดภาพใหม่ได้อย่างลื่นไหล 100% โดยไม่มีการค้าง

บทที่ 6: ตรรกะภารกิจ 3 เฟส และการหลบหลีกสิ่งกีดขวาง

6.1 ตรรกะภารกิจ 3 เฟส (3-Phase Mission Execution)

เพื่อตอบสนองต่อโจทย์การทำงานอัตโนมัติจริงในสนาม ระบบได้นำเสนอตรรกะ 3 เฟสสมบูรณ์แบบ:

ลำดับการทำงาน 3 เฟส

1. **Phase 1 (Clear Obstacles):** ตรวจสอบก่อนบล็อกที่เป็นสิ่งกีดขวางในช่องที่ไม่ได้ถูกเลือกเป็นเป้าหมาย จากนั้นสั่งให้แขนกลไปดูดบล็อกเหล่านั้นย้ายไปวางพักไว้ที่ช่อง **Feeder 1..4** ทางฝั่งซ้าย
2. **Phase 2 (Stack Goal):** สั่งแขนกลไปดูดบล็อกเป้าหมายที่เลือกไว้ตามลำดับ (#1, #2, #3, #4) นำมาวางซ้อนกันในแนวตั้งที่ช่อง **Center Goal [1, 1]** โดยบล็อกไว้สูงสุด 4 ชั้น (ความสูงจะเพิ่มขึ้นเป็น $Z = 30.0, 55.0, 80.0, 105.0$ มม.)
3. **Phase 3 (Restore Obstacles):** เมื่อวางซ้อนเป้าหมายเสร็จสิ้น แขนกลจะไปดูดบล็อกสิ่งกีดขวางจากช่อง Feeder กลับมาวางคืนยังตำแหน่งเดิมของมันในตารางกริดอย่างแม่นยำ

6.2 ระบบป้องกันการชนเสา Goal (Goal Collision Avoidance)

เมื่อบอลชนกัน 4 ชั้น เสาเป้าหมายจะสูงถึง 130 มม. หากแกนกลบืนตรง ๆ ระหว่างช่อง Feeder (ทางซ้าย $Y = +79$ มม.) กับช่องตารางแถวขวา ($Y = -35$ มม.) เส้นทางบืนตรงจะตัดผ่านกึ่งกลางเสา Goal พอดี ทำให้ชนเสาล้มทันที!

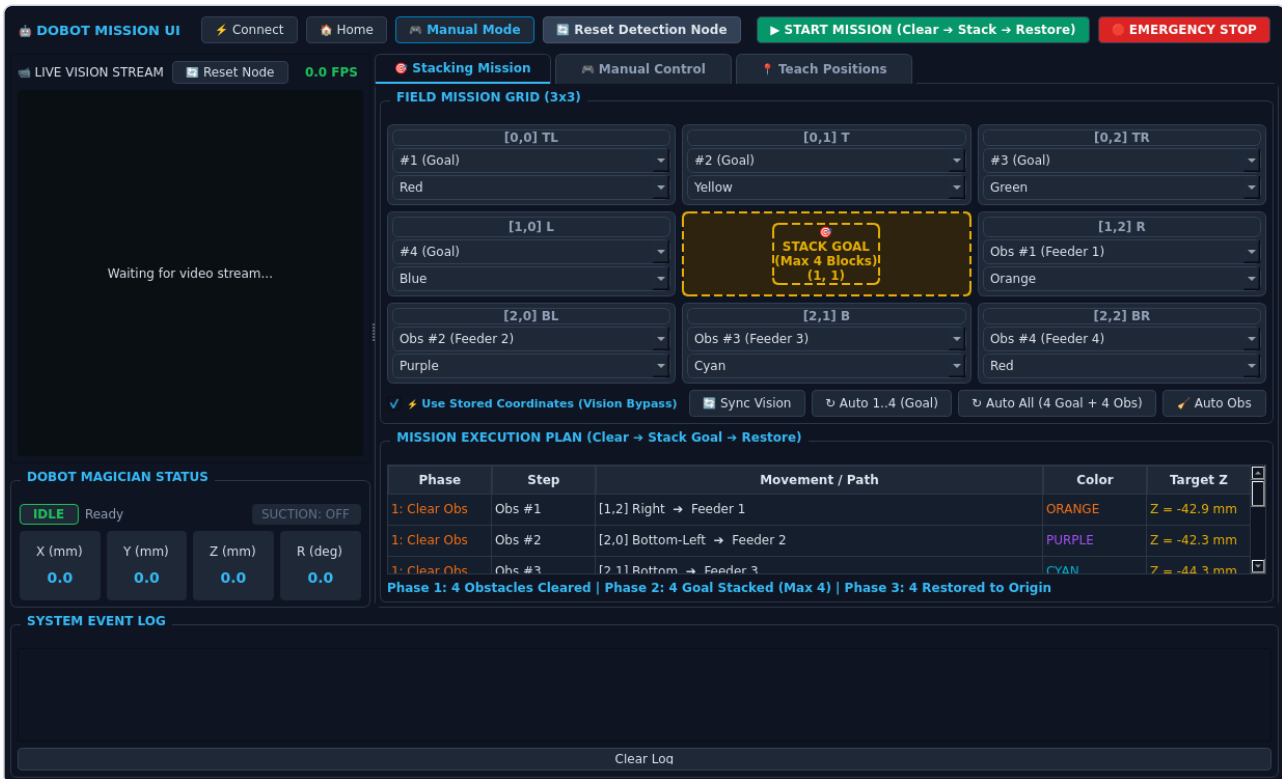
อัลกอริทึมการหาเส้นทางเลี่ยง (Path Detour Planning):

- กำหนดพื้นที่กระบอกวงห้าม (Keepout Cylinder) รัศมี 38.0 มม. รอบจุดศูนย์กลาง Goal (176.0, 0.0)
- ก่อนบืน ฟังก์ชัน `_point_to_segment_dist()` จะคำนวณระยะทางจากเส้นทางบืนตรงไปยังจุดศูนย์กลาง Goal
- หากพบว่าระยะทางสั้นกว่า 38.0 มม. (จะชนเสา) ระบบจะแทรก **จุดเลี่ยงเสา (Detour Waypoint)** อัตโนมัติ:
 - North Corridor Bypass:** อ้อมขึ้นด้านบนผ่านพิกัด $X = 222.0$ มม. (เหนือแถว 0)
 - South Corridor Bypass:** อ้อมลงด้านล่างผ่านพิกัด $X = 132.0$ มม. (ใต้แถว 2 ใกล้เคียงหุ่นยนต์)
- Safe Vertical Clearance:** เมื่อเริ่มบืนบอลวางซ้อนบน Goal ระดับความสูงบืนข้ามสนามทั้งหมดจะถูกยกขึ้นสู่ $Z = 142.0$ มม. (สูงกว่าเสา 4 ชั้นอย่างปลอดภัย) และการลง/ขึ้นที่เสา Goal จะเป็นแนวตั้ง 90° เสมอ

บทที่ 7: คู่มือการใช้งานหน้าต่างโปรแกรม (UI Walkthrough)

7.1 หน้าจอภารกิจ (Stacking Mission Tab)

แถบเมนูหลักสำหรับควบคุมภารกิจอัตโนมัติ 3 เฟส:

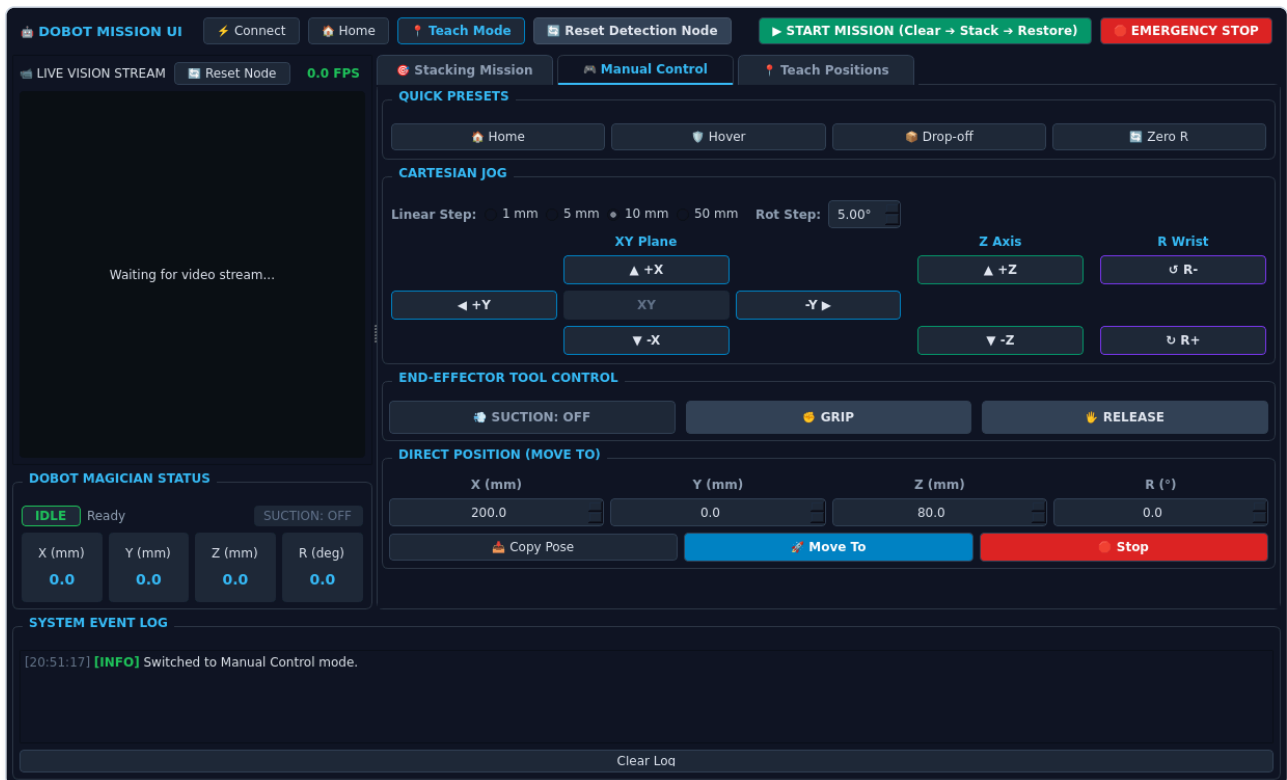


รูปที่ 1: หน้าจอหลัก Stacking Mission แสดงภาพกล้องสด, ตารางกริด 3x3 พร้อมการกำหนด Feeder และตารางแผนการทำงาน 3 เฟส

- ปุ่ม **Auto 1..4 (Goal)**: กำหนด 4 บล็อกแรกเป็น Goal ตามเข็มนาฬิกา และล้างช่องที่เหลือ
- ปุ่ม **Auto All (4 Goal + 4 Obs)**: กำหนด 4 บล็อกเป็น Goal และอีก 4 บล็อกเป็นสิ่งกีดขวางส่งไป Feeder 1..4 กันที
- ปุ่ม **Auto Obs**: กวาดบล็อกที่เหลือที่ตรวจพบสี ส่งไปลงช่อง Feeder ที่ว่างให้อัตโนมัติ
- ตาราง **MISSION EXECUTION PLAN**: แสดงขั้นตอนทั้ง 3 เฟสอย่างละเอียด (แถบสีส้ม = Phase 1 Clear, แถบสีฟ้า = Phase 2 Stack, แถบสีเขียว = Phase 3 Restore)
- ปุ่ม **START MISSION (Clear → Stack → Restore)**: สั่งคำสั่งเริ่มทำงานอัตโนมัติครบวงจร

7.2 หน้าจอควบคุมด้วยตนเอง (Manual Control Tab)

แถบเมนูสำหรับทดสอบการทำงานของแขนกล จีอกกิ้งมือ และสั่งงานเครื่องมือปลายแขน:

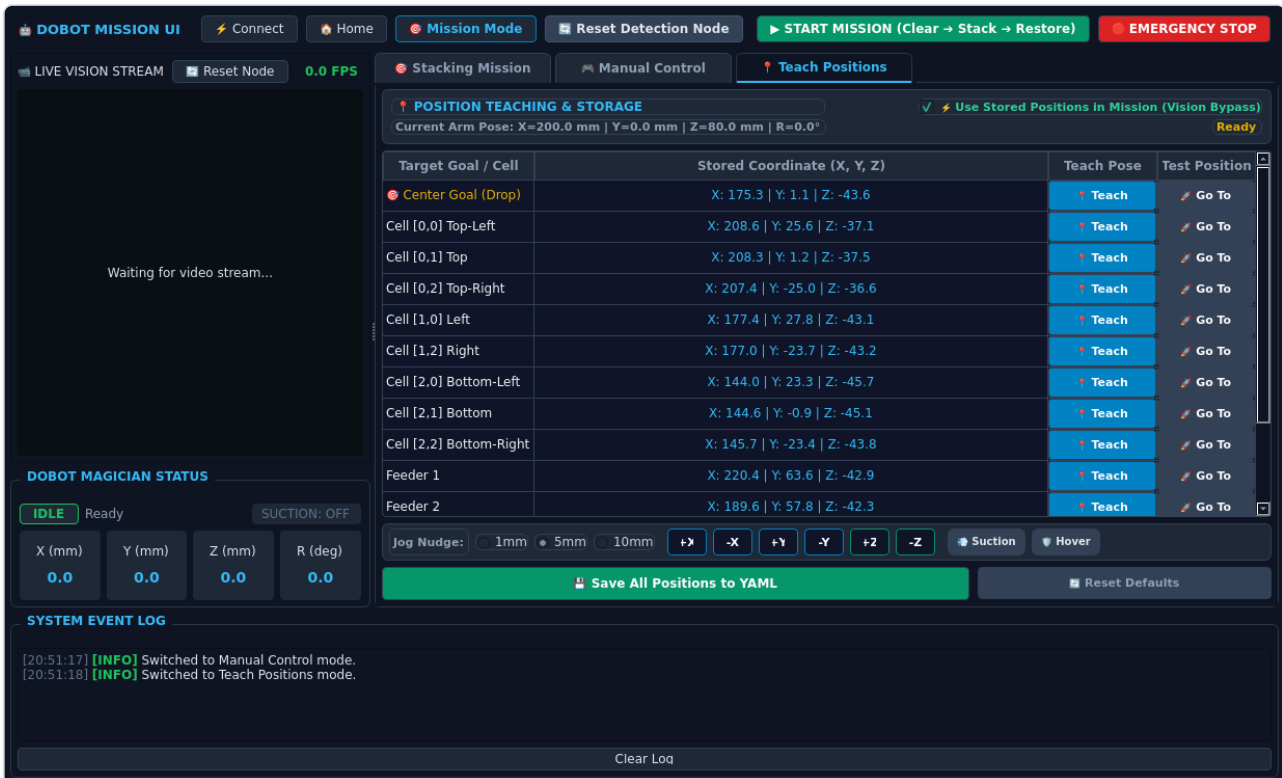


รูปที่ 2: หน้าจอ Manual Control แสดงแผงปุ่ม Quick Presets, Cartesian Jogging, Tool Control และ Direct Position

- **Quick Presets:** มินิไปยังจุดมาตรฐานทันที (`Home`, `Hover` Z=80mm, `Drop-off` Z=30mm, `Zero R`)
- **Cartesian Jog:** ปุ่มทิศทางปรับตำแหน่งแกน X, Y, Z, R เลือกขนาดก้าวได้ (1mm, 5mm, 10mm, 50mm)
- **Tool Control:** ปุ่มทดสอบเปิด/ปิดปั๊มสูญญากาศ (`SUCTION`) และสั่งหนีบ/คลายมือจับ (`GRIP` / `RELEASE`)
- **Direct Position:** พิมพ์พิกัด X, Y, Z, R ที่ต้องการ แล้วกด `🚀 Move To` หรือกด `📄 Copy Pose` เพื่อดึงพิกัดปัจจุบันมาใส่

7.3 หน้าจอบันทึกพิกัดตำแหน่ง (Teach Positions Tab)

เมนูพิเศษสำหรับบันทึกพิกัดทางกายภาพของแต่ละช่อง เสา Goal และ Feeder ลงในไฟล์ Configuration:



รูปที่ 3: หน้าจอ Teach Positions สำหรับสอนจำพิกัดแต่ละช่องเพื่อใช้งานในโหมด Vision Bypass

- ใช้จ็อบกึ่งมือช่วยหัดจุดให้แตะตรงกลางบล็อกหรือช่องเป้าหมาย
- กดปุ่ม **Teach** เพื่อบันทึกพิกัดปัจจุบันเข้าตาราง
- กดปุ่ม **Go To** เพื่อทดสอบให้แขนกลบินไปหาตำแหน่งนั้น
- กดปุ่ม **Save All Positions to YAML** เพื่อบันทึกลงไฟล์ `dobot_ui.yaml` ถาวร ทำให้ระบบทำงานต่อได้ทันทีแม้เครื่องจะใช้งานไม่ได้

บทที่ 8: คำสั่งเริ่มต้นใช้งานและการแก้ไขปัญหา

8.1 คำสั่งคอมไพล์และเปิดระบบ (Quick Start)

```
# 1. โหลด Environment และคอมไพล์แพ็คเกจ
cd /home/thxncdzch/dobot_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash

# 2. เปิดระบบทั้งหมดพร้อมกันด้วย Launch File (กล้อง + ตัวควบคุมแขนกล + GUI)
ros2 launch dobot_v2 dobot_system.launch.py

# หรือเปิดเฉพาะ GUI ในโหมดจำลอง (Mock Simulation Mode โดยไม่ต้องต่อฮาร์ดแวร์จริง)
ros2 launch Dobot_UI dobot_ui.launch.py mock:=true
```

8.2 ปัญหาที่พบบ่อยและวิธีแก้ไข (Troubleshooting)

⚠ ปัญหาการเชื่อมต่อ USB Serial บน Linux

หากต่อ Dobot แล้วขึ้น Error ไม่สามารถเปิดพอร์ต `/dev/ttyUSB0` ได้ เกิดจากสิทธิ์ผู้ใช้ Linux:

```
sudo usermod -a -G dialout $USER
sudo chmod 666 /dev/ttyUSB0
# จากนั้น Logout ออกและ Login ใหม่อีกครั้ง
```

💡 **ปัญหาหลอดค้างหรือขึ้น Device Busy**

หากหลอดเปิดไม่ติด ให้กดปุ่ม  **Reset Detection Node** บนแถบด้านบนของ GUI โปรแกรมจะทำการตัดโปรเซสเดิมและเปิดใหม่อัตโนมัติใน 0.5 วินาที

จัดทำโดย: Thanadech (Google Deepmind Pair-Programming Assistant - Antigravity) • กันยายน 2026